J2EE Practical Program and VIVA Qs

----------JDBC----------

1. **Write a program that performs CRUD operation JDBC**

MySQL Table

Run this in XAMPP phpMyAdmin:

```
CREATE DATABASE studentdb;

USE studentdb;

CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    subject VARCHAR(50)
);
```

Use this in **VS Code**

File: `JDBC_CRUD.java`

```
import java.sql.*;
import java.util.*;

public class JDBC_CRUD {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/studentdb", "root", ""
            );

            while (true) {
                System.out.println("\n--- CRUD MENU ---");
                System.out.println("1. Insert");
                System.out.println("2. Display");
                System.out.println("3. Update");
                System.out.println("4. Delete");
                System.out.println("5. Exit");
                System.out.print("Enter choice: ");
                int ch = sc.nextInt();

                if (ch == 1) {
                    System.out.print("Enter name: ");
                    String name = sc.next();
                    System.out.print("Enter subject: ");
                    String subject = sc.next();

                    PreparedStatement ps = con.prepareStatement(
                        "INSERT INTO students(name, subject) VALUES(?, ?)"
```

```
                    );
                    ps.setString(1, name);
                    ps.setString(2, subject);
                    ps.executeUpdate();
                    System.out.println("Inserted Successfully!");
                }

                else if (ch == 2) {
                    Statement st = con.createStatement();
                    ResultSet rs = st.executeQuery("SELECT * FROM
students");

                    while (rs.next()) {
                        System.out.println(rs.getInt(1) + "  " +
                                           rs.getString(2) + "  " +
                                           rs.getString(3));
                    }
                }

                else if (ch == 3) {
                    System.out.print("Enter ID to update: ");
                    int id = sc.nextInt();
                    System.out.print("Enter new name: ");
                    String name = sc.next();
                    System.out.print("Enter new subject: ");
                    String subject = sc.next();

                    PreparedStatement ps = con.prepareStatement(
                        "UPDATE students SET name=?, subject=? WHERE id=?"
                    );
                    ps.setString(1, name);
                    ps.setString(2, subject);
                    ps.setInt(3, id);
                    ps.executeUpdate();
                    System.out.println("Updated Successfully!");
                }

                else if (ch == 4) {
                    System.out.print("Enter ID to delete: ");
                    int id = sc.nextInt();

                    PreparedStatement ps = con.prepareStatement(
                        "DELETE FROM students WHERE id=?"
                    );
                    ps.setInt(1, id);
                    ps.executeUpdate();
                    System.out.println("Deleted Successfully!");
                }

                else {
                    break;
                }
            }
            con.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

----------Servlet----------

2. Write a program that performs CRUD operation servlet.

Project Structure

```
ServletCRUD/
 ├── src/
 │     └── com.demo/
 │           ├── Student.java
 │           ├── MainServlet.java
 └── webapp/
       ├── index.html
       └── update.html
```

1. Student.java (model class)

```
package com.demo;

public class Student {
    public int id;
    public String name;
    public String email;
    public String course;
    public String city;

    public Student(int id, String name, String email, String course,
String city) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
        this.city = city;
    }
}
```

2. MainServlet.java (Single Servlet CRUD)

```
package com.demo;

import java.io.*;
import java.util.ArrayList;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/crud")
public class MainServlet extends HttpServlet {

    ArrayList<Student> list = new ArrayList<>();
    int idCounter = 1;

    protected void doPost(HttpServletRequest req, HttpServletResponse
res)
            throws ServletException, IOException {
```

```
        String action = req.getParameter("action");

        // ADD
        if (action.equals("add")) {
            String name = req.getParameter("name");
            String email = req.getParameter("email");
            String course = req.getParameter("course");
            String city = req.getParameter("city");

            list.add(new Student(idCounter++, name, email, course,
city));
            res.sendRedirect("crud?action=view");
            return;
        }

        // UPDATE
        if (action.equals("update")) {
            int id = Integer.parseInt(req.getParameter("id"));
            String name = req.getParameter("name");
            String email = req.getParameter("email");
            String course = req.getParameter("course");
            String city = req.getParameter("city");

            for (Student s : list) {
                if (s.id == id) {
                    s.name = name;
                    s.email = email;
                    s.course = course;
                    s.city = city;
                    break;
                }
            }
            res.sendRedirect("crud?action=view");
        }
    }

    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
            throws ServletException, IOException {

        String action = req.getParameter("action");
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        // DELETE
        if (action.equals("delete")) {
            int id = Integer.parseInt(req.getParameter("id"));
            list.removeIf(s -> s.id == id);
            res.sendRedirect("crud?action=view");
            return;
        }

        // VIEW
        if (action.equals("view")) {

            out.println("<h1>Student List</h1>");
            out.println("<a href='index.html'>Add Student</a><br><br>");

            out.println("<table border='1'>");
```

```
out.println("<tr><th>ID</th><th>Name</th><th>Email</th><th>Course</th><t
h>City</th><th>Action</th></tr>");

            for (Student s : list) {
                out.println("<tr>");
                out.println("<td>" + s.id + "</td>");
                out.println("<td>" + s.name + "</td>");
                out.println("<td>" + s.email + "</td>");
                out.println("<td>" + s.course + "</td>");
                out.println("<td>" + s.city + "</td>");
                out.println("<td>"
                        + "<a href='update.html?id=" + s.id +
"'>Update</a> | "
                        + "<a href='crud?action=delete&id=" + s.id +
"'>Delete</a>"
                        + "</td>");
                out.println("</tr>");
            }
            out.println("</table>");
        }
    }
}
```

3. index.html (Add Student Form)

```
<!DOCTYPE html>
<html>
<body>

<h2>Add Student</h2>

<form action="crud" method="post">
    <input type="hidden" name="action" value="add">

    Name: <input type="text" name="name"><br><br>
    Email: <input type="text" name="email"><br><br>
    Course: <input type="text" name="course"><br><br>
    City: <input type="text" name="city"><br><br>

    <input type="submit" value="Add Student">
</form>

<br>
<a href="crud?action=view">View All Students</a>

</body>
</html>
```

4. update.html (Update Form)

```
<!DOCTYPE html>
<html>
<body>

<h2>Update Student</h2>

<form action="crud" method="post">
```

```
    <input type="hidden" name="action" value="update">

    ID: <input type="number" name="id"><br><br>
    New Name: <input type="text" name="name"><br><br>
    New Email: <input type="text" name="email"><br><br>
    New Course: <input type="text" name="course"><br><br>
    New City: <input type="text" name="city"><br><br>

    <input type="submit" value="Update Student">
</form>

</body>
</html>
```

3. **Write a servlet that displays a "Welcome" message if the user is visiting the site for the first time, and "Welcome back" if they have visited before. (Using Cookie)**

SERVLET FILE — prg36.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/prg36")
public class prg36 extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String username = request.getParameter("username");
        boolean isReturningUser = false;

        // Get all cookies sent by the browser
        Cookie[] cookies = request.getCookies();

        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("user_" + username)
                        && c.getValue().equals("visited")) {
                    isReturningUser = true;
                    break;
                }
            }
        }

        // Validate username
        if (username == null || username.trim().isEmpty()) {
            out.println("<p>Please enter a valid name.</p>");
            out.println("<a href='index36.html'>Go Back</a>");
        } else {
```

```
            if (isReturningUser) {
                out.println("<h2>Welcome back, " + username + "!</h2>");
            } else {
                out.println("<h2>Welcome " + username + ", this is your
first visit!</h2>");

                // Create cookie to remember user for future visits
                Cookie userCookie = new Cookie("user_" + username,
"visited");
                userCookie.setMaxAge(60 * 60 * 24 * 30);  // 30 days
                response.addCookie(userCookie);
            }
        }
    }
}
```

HTML File — index36.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Cookie Example</title>
</head>
<body>

<h2>Enter your name</h2>

<form action="prg36" method="post">
    <label for="username">Name:</label>
    <input type="text" name="username" required>
    <br><br>
    <input type="submit" value="Submit">
</form>

</body>
</html>
```

4. **Write a Java Servlet application that authenticates users using a login form. Upon successful login, store the user's information (like username) in the HTTP session. Display a personalized welcome page using session data. For the unsuccessful attempt display appropriate message. Also, implement a logout function that invalidates the session.**

SERVLET 1 — prg40 (Login Servlet)

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/prg40")
```

```
public class prg40 extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        String usnm = "admin";
        String pasw = "1234";

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (usnm.equals(username) && pasw.equals(password)) {
            HttpSession session = request.getSession();
            session.setAttribute("username", username);
            response.sendRedirect("prg40B");
        } else {
            out.println("<h3>Invalid Login!</h3>");
            out.println("<a href='index40.html'>Try Again</a>");
        }
    }
}
```

SERVLET 2 — prg40A (Logout Servlet)

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/prg40A")
public class prg40A extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();  // end session
        }

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h3>You have been logged out.</h3>");
        out.println("<a href='index40.html'>Login Again</a>");
    }
}
```

SERVLET 3 — prg40B (Welcome Page + Session Check)

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/prg40B")
public class prg40B extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession(false);
        String username = null;

        if (session != null) {
            username = (String) session.getAttribute("username");
        }

        if (username != null) {
            out.println("<h2>Welcome, " + username + "!</h2>");
            out.println("<a href='prg40A'>Logout</a>");
        } else {
            out.println("<h3>You are not logged in.</h3>");
            out.println("<a href='index40.html'>Login Here</a>");
        }
    }
}
```

HTML FILE — index40.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Login Page</title>
</head>
<body>

<h2>Login</h2>

<form action="prg40" method="post">

    <label>Username:</label>
    <input type="text" name="username" required>
    <br><br>

    <label>Password:</label>
    <input type="password" name="password" required>
    <br><br>
```

```
    <input type="submit" value="Login">

</form>
</body>
</html>
```

5. **Write a servlet that displays a "Welcome" message if the user is visiting the site for the first time, and "Welcome back" if they have visited before. (Using Cookie)**

Servlet Code (prg36.java)

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/prg36")
public class prg36 extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String username = request.getParameter("username");
        boolean isReturningUser = false;

        // Read cookies
        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie c : cookies) {
                if (c.getName().equals("user_" + username) &&
c.getValue().equals("visited")) {
                    isReturningUser = true;
                    break;
                }
            }
        }

        // Validation
        if (username == null || username.trim().isEmpty()) {
            out.println("<p>Please enter a valid name.</p>");
            out.println("<a href='index36.html'>Go Back</a>");
        }
        else {
            // Check returning or first-time
            if (isReturningUser) {
                out.println("<h2>Welcome back, " + username + "!</h2>");
            }
            else {
```

```
                out.println("<h2>Welcome " + username + ", this is your
first visit!</h2>");

                // Create cookie for new user
                Cookie userCookie = new Cookie("user_" + username,
"visited");
                userCookie.setMaxAge(60 * 60 * 24 * 30); // 30 days
                response.addCookie(userCookie);
            }
        }
    }
}
```

✅ Correct HTML File (index36.html)

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>User Visit Tracker</title>
</head>
<body>

    <h2>Enter your name</h2>

    <form action="prg36" method="post">
        <label for="username">Name:</label>
        <input type="text" name="username" required>
        <input type="submit" value="Submit">
    </form>

</body>
</html>
```

----------JSP----------

6. Write a program that performs CRUD operation JSP

JSP CRUD — Design

```
webapp/
 ├── main.jsp      ← EVERYTHING (Add + View + Delete)
 ├── add.jsp       ← Add Form
 ├── update.jsp    ← Update Form
src/
 └── Student.java  ← simple model
```

1. Student.java (VERY SIMPLE MODEL)

(Needed to store objects. Goes in `src/com.demo/Student.java`)

```
package com.demo;

public class Student {
    public int id;
    public String name;
    public String email;
    public String course;
```

```
    public Student(int id, String name, String email, String course) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
    }
}
```

2. add.jsp (Simple Add Form)

```
<!DOCTYPE html>
<html>
<body>

<h2>Add Student</h2>

<form action="main.jsp" method="post">
    <input type="hidden" name="action" value="add">

    Name: <input type="text" name="name"><br><br>
    Email: <input type="text" name="email"><br><br>
    Course: <input type="text" name="course"><br><br>

    <input type="submit" value="Add Student">
</form>

<br><a href="main.jsp">View Students</a>

</body>
</html>
```

3. update.jsp (Simple Update Form)

```
<%@ page import="java.util.*, com.demo.Student" %>
<!DOCTYPE html>
<html>
<body>

<%
    int id = Integer.parseInt(request.getParameter("id"));
    ArrayList<Student> list = (ArrayList<Student>)
session.getAttribute("list");
    Student stu = null;

    for (Student s : list) {
        if (s.id == id) {
            stu = s;
            break;
        }
    }
%>

<h2>Update Student</h2>

<form action="main.jsp" method="post">
    <input type="hidden" name="action" value="update">
    <input type="hidden" name="id" value="<%= stu.id %>">
```

```
    Name: <input type="text" name="name" value="<%= stu.name
%>"><br><br>
    Email: <input type="text" name="email" value="<%= stu.email
%>"><br><br>
    Course: <input type="text" name="course" value="<%= stu.course
%>"><br><br>

    <input type="submit" value="Update">
</form>

</body>
</html>
```

4. main.jsp (THE MAIN CRUD FILE — SUPER SIMPLE)

```
<%@ page import="java.util.*, com.demo.Student" %>

<%! int counter = 1; %>

<%
    // Create list if not exists
    ArrayList<Student> list = (ArrayList<Student>)
session.getAttribute("list");
    if (list == null) {
        list = new ArrayList<Student>();
        session.setAttribute("list", list);
    }

    String action = request.getParameter("action");

    // ADD OPERATION
    if ("add".equals(action)) {
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        list.add(new Student(counter++, name, email, course));
    }

    // DELETE OPERATION
    if ("delete".equals(action)) {
        int id = Integer.parseInt(request.getParameter("id"));
        list.removeIf(s -> s.id == id);
    }

    // UPDATE OPERATION
    if ("update".equals(action)) {
        int id = Integer.parseInt(request.getParameter("id"));
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        for (Student s : list) {
            if (s.id == id) {
                s.name = name;
                s.email = email;
                s.course = course;
                break;
            }
        }
    }
```

```
%>

<!DOCTYPE html>
<html>
<body>

<h1>Student Records</h1>

<a href="add.jsp">Add Student</a><br><br>

<table border="1" cellpadding="5">
    <tr>

<th>ID</th><th>Name</th><th>Email</th><th>Course</th><th>Action</th>
    </tr>

<%
    for (Student s : list) {
%>
    <tr>
        <td><%= s.id %></td>
        <td><%= s.name %></td>
        <td><%= s.email %></td>
        <td><%= s.course %></td>
        <td>
            <a href="update.jsp?id=<%= s.id %>">Update</a> |
            <a href="main.jsp?action=delete&id=<%= s.id %>">Delete</a>
        </td>
    </tr>
<%
    }
%>

</table>

</body>
</html>
```

7. **Write a JSP program that allows the user to log in by entering their username (login.jsp). Once the form is submitted, store the username in the session object and display a personalized welcome message on a separate JSP page (welcome.jsp). The application should retain the username across multiple pages using the session and also provide an option to logout (logout.jsp), which will invalidate the session and redirect the user back to the login page (login.jsp). This program demonstrates how session tracking is used to manage user state across multiple JSP pages.**

File: `Login.jsp`

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Login Page</title>
</head>
<body>
```

```
<h2>Login</h2>

<form action="Welcome.jsp" method="post">
    Username: <input type="text" name="username" required />
    <input type="submit" value="Login" />
</form>

<%
    String message = request.getParameter("msg");
    if (message != null) {
%>
        <p style="color:red;"><%= message %></p>
<%
    }
%>

</body>
</html>
```

File: `Welcome.jsp`

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Welcome Page</title>
</head>
<body>

<%
    String username = request.getParameter("username");

    // Save username in session only if passed from login
    if (username != null && !username.trim().isEmpty()) {
        session.setAttribute("username", username);
    }

    String user = (String) session.getAttribute("username");

    // If user not logged in → redirect to login page
    if (user == null) {
        response.sendRedirect("Login.jsp?msg=Please+login+first");
        return;
    }
%>

<h2>Welcome, <%= user %>!</h2>
<a href="Logout.jsp">Logout here</a>

</body>
</html>
```

File `Logout.jsp`

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
```

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Logout Page</title>
</head>
<body>

<%
    session.invalidate(); // End the session completely
    response.sendRedirect("Login.jsp?msg=You+have+been+logged+out");
%>

</body>
</html>
```

8. **Write a JSP that displays a "Welcome" message if the user is visiting the site for the first time, and "Welcome back" if they have visited before. (Using cookie)**

JSP Code (Cookie Example)

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Cookie Example</title>
</head>
<body>

<%
    boolean visit = false;

    // Read cookies
    Cookie[] cookies = request.getCookies();
    if (cookies != null) {
        for (Cookie c : cookies) {
            if ("visited".equals(c.getName())) {
                visit = true;
                break;
            }
        }
    }

    // If first time -> create cookie
    if (!visit) {
        Cookie newUser = new Cookie("visited", "true");
        newUser.setMaxAge(60 * 60 * 24);  // Valid for 1 day
        response.addCookie(newUser);
    }
%>

<h2>
<%
    if (visit) {
        out.println("Welcome Back!");
    } else {
        out.println("Welcome, You Are Visiting for the First Time!");
```

```
    }
%>
</h2>

</body>
</html>
```

---------VIVA QUESTIONS WITH ANSWERS----------

UNIT–1: Introduction to J2EE & JDBC

---

1. What is J2EE?

J2EE (Java 2 Enterprise Edition) is a platform for building **enterprise-level web applications** using Java technologies like Servlets, JSP, EJB, JDBC, etc.

---

2. Difference between Java SE and Java EE?

Java SE (Standard Edition)	Java EE (Enterprise Edition)
Core Java	Enterprise Java
Desktop applications	Web applications
OOP concepts	Servlets, JSP, EJB, JDBC

---

3. What is Two-tier architecture?

Architecture with **client + database** only.
Example: JDBC standalone program.

---

4. What is Three-tier architecture?

Layers:

1. Client
2. Server (Servlet/JSP)
3. Database
   Most common in web applications.

---

5. What is N-tier architecture?

More than 3 layers (Presentation → Business → Data Access → Database).

6. What is Enterprise Architecture?

A way to design large applications using layered structure for scalability.

---

7. What is Web Container?

A runtime environment that runs Servlets and JSP.
Example: **Apache Tomcat**.

---

8. Name J2EE APIs.

Servlet, JSP, EJB, JMS, JavaMail, JSF, JNDI.

---

9. What is JDBC?

Java API to connect Java applications with databases.

---

10. JDBC Architecture?

Application → JDBC Driver → Database.

---

11. Types of JDBC Drivers?

1. Type 1: JDBC-ODBC
2. Type 2: Native API
3. Type 3: Network Protocol
4. Type 4: Pure Java Driver (most common)

---

12. What is DriverManager?

Class used to load JDBC drivers and open database connections.

---

13. Statement vs PreparedStatement

Statement: For static SQL queries
PreparedStatement: For dynamic queries and prevents SQL injection

---

14. What is CallableStatement?

Used to call stored procedures in database.

---

15. What is CRUD?

Create, Read, Update, Delete operations.

UNIT–2: SERVLET

16. What is a Servlet?

A Java class that handles **HTTP requests** and generates dynamic output.

---

17. Servlet life cycle.

1. `init()`
2. `service()`
3. `destroy()`

---

18. What is web.xml?

Deployment descriptor used to configure servlets.

---

19. What is @WebServlet?

Annotation used to map servlet URL patterns without web.xml.

---

20. doGet() vs doPost().

doGet()	doPost()
Sends data through URL	Sends data in request body
Limited data	Unlimited data
Less secure	More secure

---

21. What is HttpServletRequest?

Used to read client data like form values.

---

22. What is HttpServletResponse?

Used to send output (HTML, redirect) to client.

---

23. How to redirect in Servlet?

```
response.sendRedirect("page.html");
```

---

24. What is Session Tracking?

Technique to maintain user data across multiple requests.

---

25. Name session tracking techniques.

1. URL rewriting
2. Hidden form fields
3. Cookies
4. HttpSession

---

26. What is a Cookie?

Small data stored on client browser.

---

27. What is HttpSession?

Server-side session for storing user-related data.

UNIT–3: JSP

28. What is JSP?

Java Server Pages – used to create dynamic HTML pages easily.

---

29. JSP vs Servlet?

Servlet: Java code with HTML
JSP: HTML with Java code
JSP is better for designing UI.

---

30. JSP life cycle.

JSP → converted to Servlet → compiled → init → service → destroy.

---

31. JSP Directives.

- page
- include
- taglib

---

32. Scripting elements.

- Declaration <%! %>
- Scriptlet <% %>
- Expression <%= %>

---

33. JSP Action Tags.

jsp:include, jsp:forward, jsp:param

---

34. Implicit objects in JSP.

request, response, out, session, application, config, page, pageContext

---

35. JSP include vs forward.

include	forward
Adds content	Forwards request
Happens at runtime	Stops current page

---

36. How to set session value in JSP?

```
session.setAttribute("name", "John");
```

---

37. How to read cookie in JSP?

```
Cookie[] c = request.getCookies();
```

---

38. What is EL in JSP?

Expression Language → `${}`
Used to replace Java scriptlets.

---

39. What is JSTL?

JSP Standard Tag Library used for loops, conditions, SQL operations.

UNIT–4: EJB, MVC, Hibernate

40. What is EJB?

Enterprise Java Beans – server-side reusable components for business logic.

---

41. Types of EJB?

1. Session Bean
2. Entity Bean
3. Message Driven Bean

---

42. What is MVC?

Model → data & logic
View → UI
Controller → request handler

---

43. Why MVC?

Better separation of code and easier maintenance.

44. What is Hibernate?

An ORM tool that maps Java objects to database tables.

---

45. What is ORM?

Object Relational Mapping – converts objects to relational DB tables.

---

46. What is Configuration file in Hibernate?

`hibernate.cfg.xml`

---

47. What is Mapping file?

Maps Java class fields to database columns.

---

48. What is SessionFactory?

Creates and manages Hibernate Sessions.

---

49. What is HQL?

Hibernate Query Language – object-oriented version of SQL.

UNIT–5: Spring Framework & Spring Boot

50. What is Spring Framework?

A Java framework for developing flexible enterprise applications.

---

51. What is Dependency Injection (DI)?

Spring injects required objects into a class automatically.

---

52. What is Inversion of Control (IoC)?

Spring container controls object creation instead of the programmer.

---

53. Bean lifecycle?

Create → Set properties → Init → Use → Destroy.

---

54. What is ApplicationContext?

Advanced Spring container.

---

55. What is Spring MVC?

Framework based on MVC for building web applications.

---

56. What is DispatcherServlet?

The front controller in Spring MVC.

---

57. What is AOP?

Aspect-Oriented Programming – handles cross-cutting concerns like logging.

---

58. What is Spring Boot?

Spring framework with auto-configuration and embedded servers.

---

59. Why use Spring Boot?

- No XML
- Embedded Tomcat
- Fast development

---

60. What is @SpringBootApplication?

Enables configuration + component scan + auto-configuration.

---

61. What is @RestController?

Used to create REST APIs.

---

62. What is @RequestMapping?

Maps URL to controller method.

---

63. What is @ExceptionHandler?

Handles exceptions inside controller class.

---

64. What is Spring Data JPA?

Spring framework for easy database operations using Repository.

---

65. CRUD in Spring Boot?

With JpaRepository:

- save()
- findAll()
- delete()
- findById()

EXTRA IMPORTANT QUESTIONS

66. forward() vs sendRedirect()

forward()	sendRedirect()
Server side	Client side
URL unchanged	URL changes
Faster	Slower

---

67. What is WAR file?

Web Application Archive used for deployment in servers like Tomcat.

---

68. How to deploy project in Tomcat?

Copy WAR → `tomcat/webapps` → start server.

---

69. What is a Filter in J2EE?

Used for logging, validation, authentication, etc.

---

70. What is Listener in J2EE?

Tracks application events like session creation or destruction.